

Task 2 Solving with LLMs

1. I tried generating the first prompt from the chatgpt, which looks something like this:

"You will be given exactly one puzzle in JSON. The puzzle has:

- *"problem_id": a string id (copy it to the answer).*
- *"initial_string": the starting string.*
- *"transitions": an ordered list of transition objects, each with "src" and "tgt" strings.*

Task:

Transform the initial_string to the empty string ("") by repeatedly applying transitions. Each step you must:

- *Choose one transition index (0-based indexing).*
- *Find the FIRST (leftmost) occurrence of the transition's "src" inside the current string (if "src" is empty treat it as matching at position 0).*
- *Replace only that single occurrence with the transition's "tgt".*
- *Repeat until the current string becomes the empty string.*

Constraints and rules:

1. *Indices are 0-based. The solution is the ordered list of transition indices you applied.*
2. *Output must be **valid JSON only**, and nothing else (no explanation, no code fences):*

```
{
  "problem_id": "<same-as-input>",
  "solution": [<list of integer indices>]
}
```

3. *If you cannot find a sequence that empties the string after at most 500 steps, set the "solution" to an empty list [] (this will be treated as "unsolved" by the evaluator).*
4. *Do NOT output any text before or after the JSON. The JSON must be the only characters in your reply.*
5. *Always ensure indices are within the range [0, len(transitions)-1]. Do not invent indices.*

Example input:

```
{
  "problem_id": "006",
  "initial_string": "i%7&lX7%%77",
  "transitions": [
    { "src": "&lX", "tgt": "%%" },
    { "src": "%%", "tgt": "7" },
    { "src": "7", "tgt": "ii" },
    { "src": "i", "tgt": "" }
  ]
}
```

(You will be given a similar JSON. Produce the JSON answer described above and nothing else.)"

2. Which I first pasted in the chatgpt online (incognito tab) version. Because of the unsigned version nowadays runs on chatgpt 5.0 version, it possesses very good reasoning skills, in fact it gave optimised output to this problem:

```
{
  "problem_id": "009",
  "initial_string": "QfQffQQQfQfQ",
  "transitions": [
    { "src": "Q", "tgt": "Qf" },
    { "src": "Q", "tgt": "Q" },
    { "src": "Q", "tgt": "QQ" },
    { "src": "Q", "tgt": "" },
    { "src": "f", "tgt": "QQ" },
    { "src": "f", "tgt": "" },
    { "src": "f", "tgt": "ff" }
  ]
}
```

```
{
  "problem_id": "009",
  "solution": [3,5,3,5,3,3,3]
}
```

Whose real solution was this: {"problem_id": "009", "solution": [5, 3, 3, 3, 3, 5, 5, 5, 3, 5, 3, 5, 3]} from the puzzle generator by back generation/transformation-chain style and deletion (basically brute force during the puzzle generation).

3. Now let's test on the playground models first I used llama 8b Instruct model:

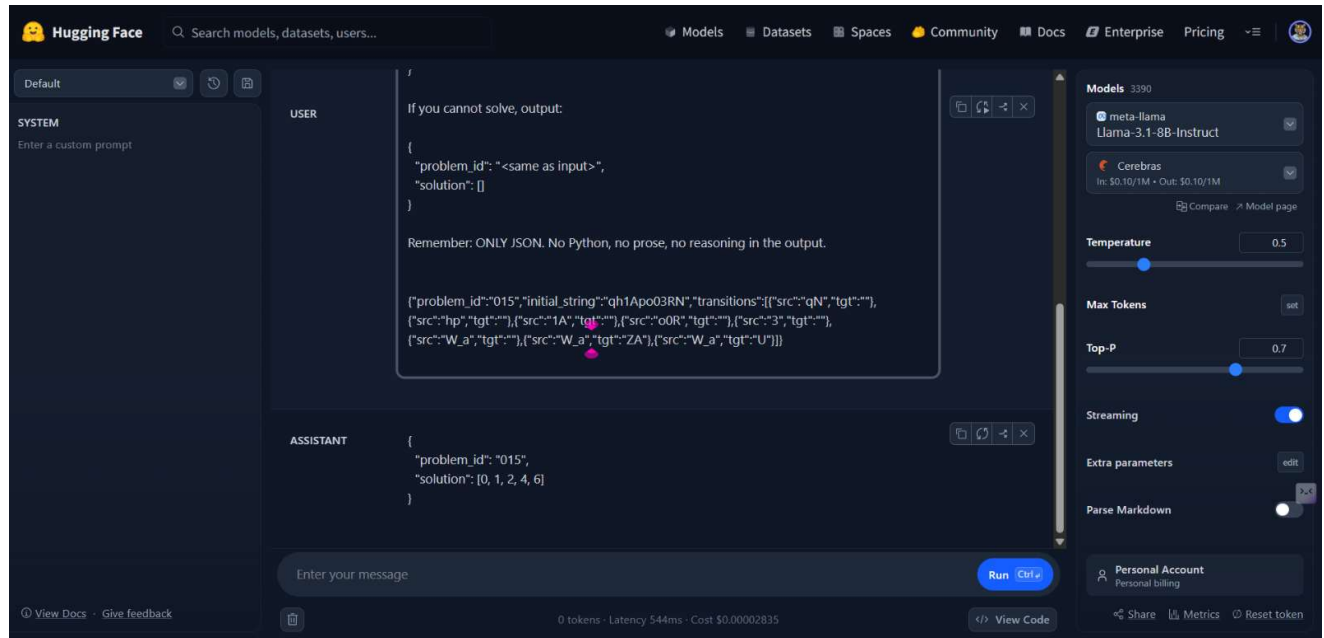
- Zero-Prompting: This was the Prompt I used for Zero Shot prompting:
 - i. *"You are given a puzzle JSON with "problem_id", "initial_string", and ordered "transitions" (each with "src" and "tgt").*
 - ii. *Apply transitions (leftmost match only) until the string becomes empty. Output ****only**** JSON:*
 - iii.
 - iv. *{"problem_id": "<same-id>", "solution": [<o-based indices>]}*
 - v.
 - vi. *If you can't reach empty within 500 steps, output "solution": []."*

This gave an abrupt python code which we call as hallucination or didn't understood the assignment to be precise.

- Going forward I gave strict prompt like this:
 - i. *"You are solving a string-rewrite puzzle. The puzzle is given as JSON with:*
 - ii. *- "problem_id": identifier*
 - iii. *- "initial_string": starting string*
 - iv. *- "transitions": a list of objects {"src": "...", "tgt": "..."}"*
 - v.
 - vi. *Task:*
 - vii. *Apply transitions step by step until the string becomes empty "".*
 - viii.
 - ix. *Rules:*
 - x. *1. At each step, pick one transition index (o-based).*
 - xi. *2. Replace the FIRST occurrence of "src" with "tgt".*
 - xii. *3. Keep going until the string is "".*
 - xiii. *4. The solution is the list of transition indices in the order applied.*
 - xiv.
 - xv. *Important:*
 - xvi. *- Do NOT explain, do NOT write code, do NOT output text.*
 - xvii. *- Output must be JSON only. Nothing before, nothing after.*
 - xviii. *- JSON format required:*
 - xix.
 - xx. *{*
 - xxi. *"problem_id": "<same as input>",*
 - xxii. *"solution": [list of integers]*
 - xxiii. *}*
 - xxiv.
 - xxv. *If you cannot solve, output:*
 - xxvi.
 - xxvii. *{*
 - xxviii. *"problem_id": "<same as input>",*
 - xxix. *"solution": []*
 - xxx. *}*
 - xxxi.
 - xxxii. *Remember: ONLY JSON. No Python, no prose, no reasoning in the output."*

- And the question was
 - i. `{"problem_id": "015", "initial_string": "qh1Apo03RN", "transitions": [{"src": "qN", "tgt": ""}, {"src": "hp", "tgt": ""}, {"src": "1A", "tgt": ""}, {"src": "o0R", "tgt": ""}, {"src": "3", "tgt": ""}, {"src": "W_a", "tgt": ""}, {"src": "W_a", "tgt": "ZA"}, {"src": "W_a", "tgt": "U"}]}`
 - ii. To which the real answer was: `{"problem_id": "015", "solution": [4, 3, 2, 1, 0]}`
 - iii. But model gave wrong answer to that puzzle: The model definitely didn't give whole wrong answer, as it left 3 and few other puzzles empty without attending in the string.

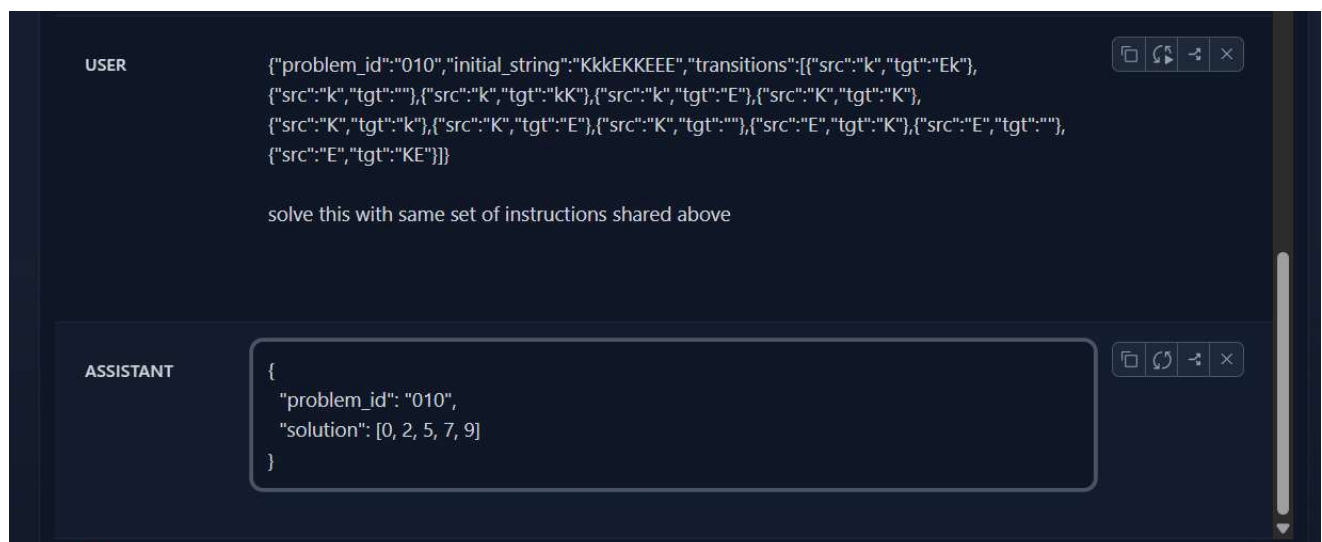
1. This was from easy dataset question:



2. Few more question were tested on the zero-shot prompting: questions and outputs are attached:

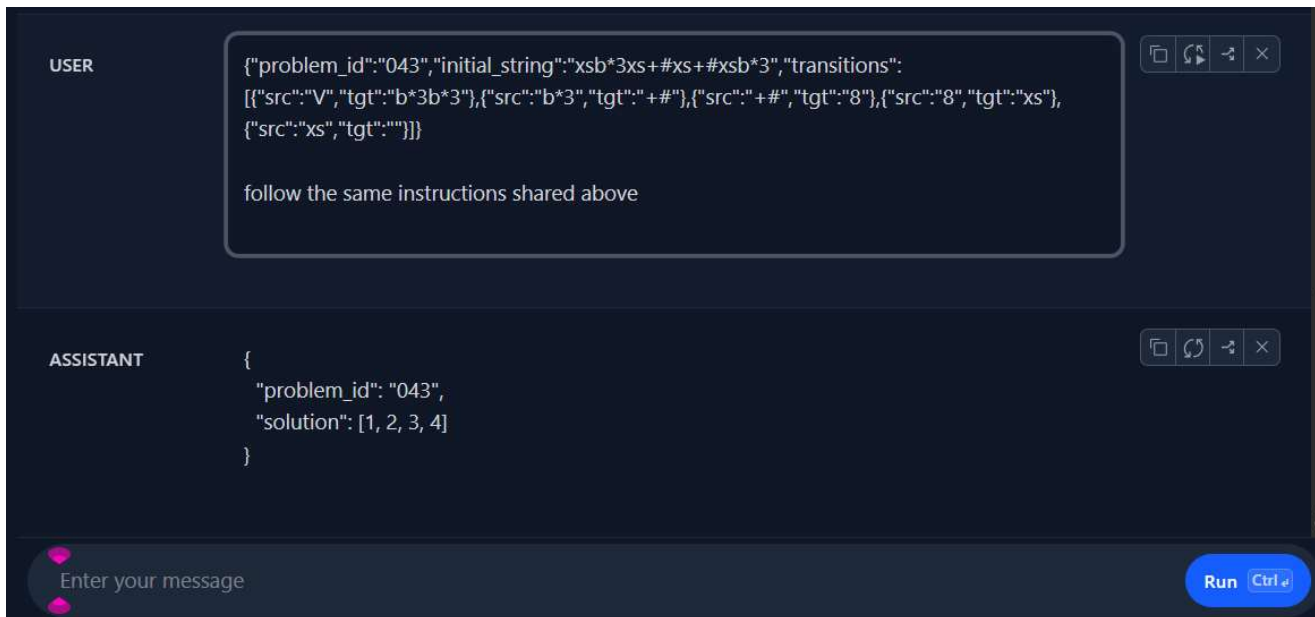
a. From medium (multi-token selection) dataset:

- i. Here again the model gave wrong answer the correct answer was: `{"problem_id": "010", "solution": [1, 7, 9, 9, 7, 9, 1, 9]}`



b. From hard (Traversing string replacing) dataset:

- i. Again the solution was `{"problem_id": "043", "solution": [4, 1, 2, 3, 4, 4, 2, 3, 4, 4, 2, 3, 4, 4, 1, 2, 3, 4]}` but model hallucinated and spent any time computing the logic for this problem as well.



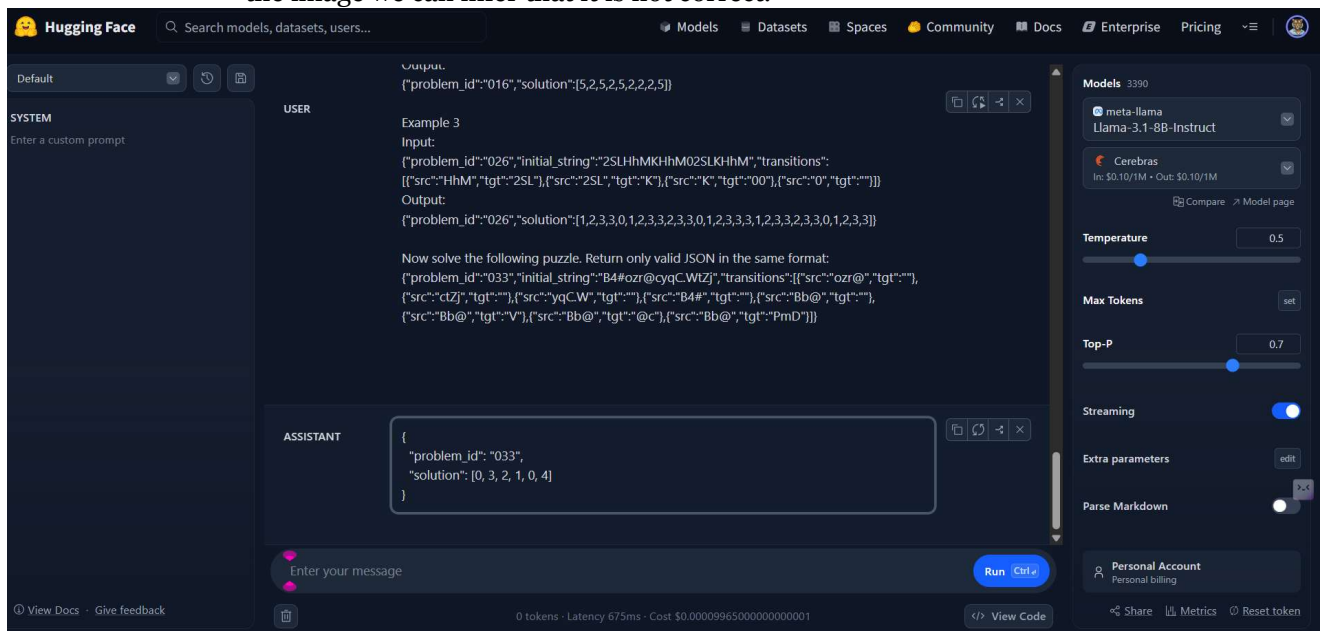
4. Now one model **LLaMa3.1 8b Instruct** is tested which couldn't solve any puzzles, I do not think testing it on further smaller models would make any sense as it would perform poorly only.
5. This brings me to sense of understanding that model with smaller parameters cannot possess good **zero-shot reasoning skills**.

6. Few Shot Prompting: The prompt that I used was:

- i. *"You are solving string-rewrite puzzles. Each puzzle is given as JSON:*
- ii. *- "problem_id": identifier*
- iii. *- "initial_string": starting string*
- iv. *- "transitions": a list of rules {"src": "...", "tgt": "..."}*
- v.
- vi. *Task:*
- vii. *Apply transitions step by step until the string becomes "".*
- viii.
- ix. *Rules:*
 - x. *1. At each step, pick one transition index (0-based).*
 - xi. *2. Replace the FIRST occurrence of "src" with "tgt".*
 - xii. *3. Continue until the string is "".*
 - xiii. *4. The solution is the list of transition indices in the order applied.*
- xiv.
- xv. *Important:*
- xvi. *- Output must be JSON only, exactly in the format shown.*
- xvii. *- Do NOT output code, reasoning, or explanations.*
- xviii. *- Do NOT output anything before or after the JSON.*
- xix.
- xx. *Examples:*
- xxi.
- xxii. *Example 1*
- xxiii. *Input:*
- xxiv. *{ "problem_id": "004", "initial_string": "buXfTof6OPg1umMKPqL#C4Dn", "transition s": [{"src": "ML#", "tgt": ""}, {"src": "C4Dn", "tgt": ""}, {"src": "buXOP", "tgt": ""}, {"src": "u m", "tgt": ""}, {"src": "fTof6", "tgt": ""}, {"src": "KPq", "tgt": ""}, {"src": "g1", "tgt": ""}, {"src": " "}*

- : "wF7", "tgt": "I"}, {"src": "wF7", "tgt": ""}, {"src": "O1nn", "tgt": "xXK"}, {"src": "O1nn", "tgt": ""}, {"src": "O1nn", "tgt": "v5c"}, {"src": "u", "tgt": ""}, {"src": "u", "tgt": "4"}]}
- xxv. Output:
- xxvi. {"problem_id": "004", "solution": [6, 5, 4, 3, 2, 1, 0]}
- xxvii.
- xxviii. Example 2
- xxix. Input:
- xxx. {"problem_id": "016", "initial_string": "...MM..MM", "transitions": [{"src": ".", "tgt": ".."}, {"src": ".", "tgt": "."}, {"src": ".", "tgt": ""}, {"src": "M", "tgt": ".."}, {"src": "M", "tgt": "MM"}, {"src": "M", "tgt": ""}, {"src": "M", "tgt": "M"}]}
- xxxi. Output:
- xxxii. {"problem_id": "016", "solution": [5, 2, 5, 2, 5, 2, 2, 5]}
- xxxiii.
- xxxiv. Example 3
- xxxv. Input:
- xxxvi. {"problem_id": "026", "initial_string": "2SLHhMKHhMo2SLKHhM", "transitions": [{"src": "HhM", "tgt": "2SL"}, {"src": "2SL", "tgt": "K"}, {"src": "K", "tgt": "oo"}, {"src": "o", "tgt": ""}]}
- xxxvii. Output:
- xxxviii. {"problem_id": "026", "solution": [1, 2, 3, 3, 0, 1, 2, 3, 3, 2, 3, 3, 0, 1, 2, 3, 3, 3, 1, 2, 3, 3, 2, 3, 3, 0, 1, 2, 3, 3]}

- Firstly, let's try easy puzzle:
 - The solution to which was {"problem_id": "033", "solution": [3, 2, 1, 0]}, but again from the image we can infer that it is not correct.



- This was easy puzzle dataset questions, which again makes sense to skip the few shot prompting for medium and harder questions and let's jump onto Chain of Thoughts puzzle.

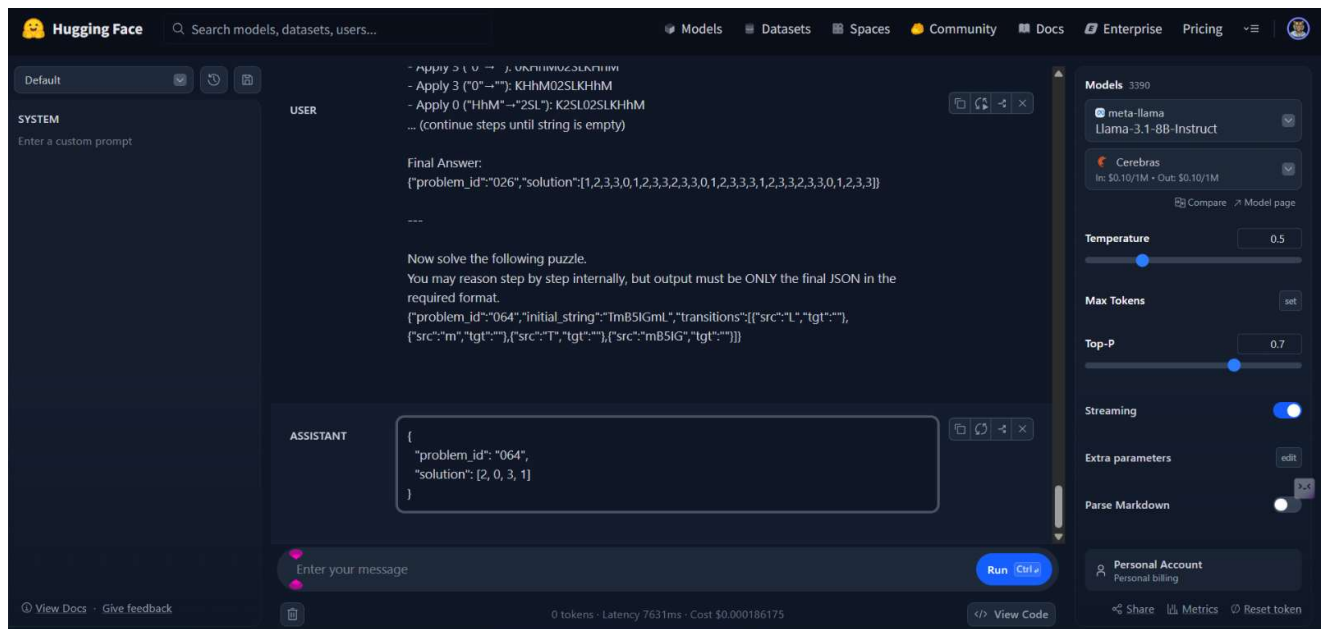
7. Chain of Thoughts prompting: The prompt I used was this:

- "You are solving string-rewrite puzzles.
- Each puzzle is given as JSON with:
- "problem_id": identifier
- "initial_string": starting string
- "transitions": list of rules {"src": "...", "tgt": "..."}.
- Task:
- Transform the "initial_string" step by step until it becomes empty "".

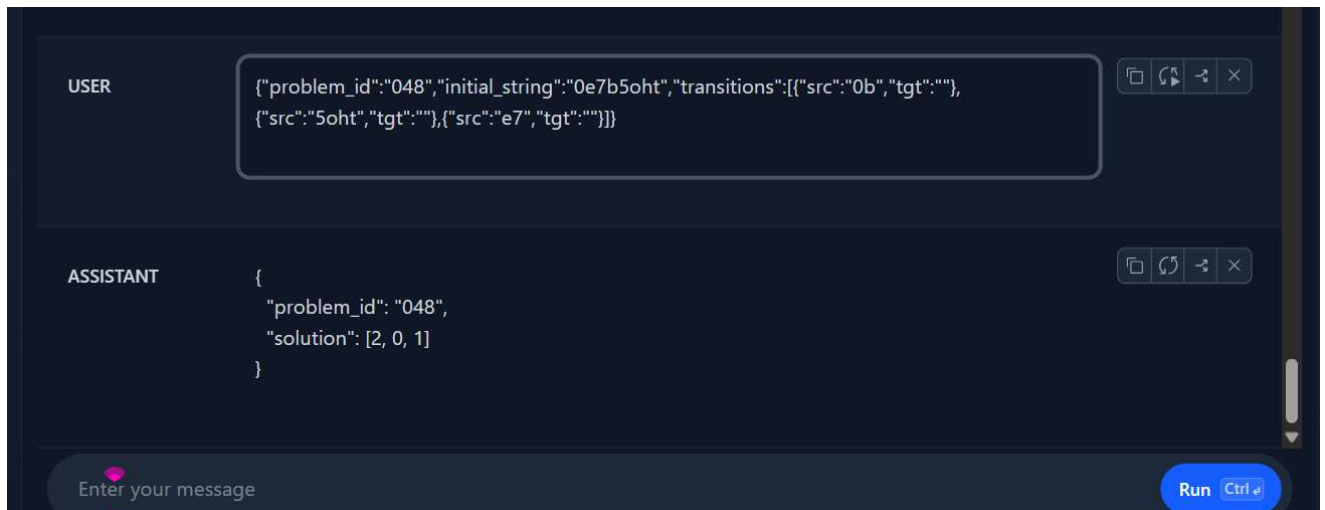
- ix.
- x. Rules:
- xi. 1. At each step, choose one transition index (0-based).
- xii. 2. Replace the FIRST occurrence of "src" with "tgt".
- xiii. 3. Continue until the string becomes "".
- xiv. 4. Record the sequence of indices used.
- xv.
- xvi. --- IMPORTANT ---
- xvii. - While solving, you may think step by step to explain how the string changes.
- xviii. - But at the end, you MUST output ONLY the final JSON result, nothing else.
- xix. - The final JSON format must be exactly:
- xx.
- xxi. {
- xxii. "problem_id": "<id>",
- xxiii. "solution": [list of indices]
- xxiv. }
- xxv.
- xxvi. If no solution is possible, output "solution": [].
- xxvii.
- xxviii. ---
- xxix.
- xxx. ### Example 1
- xxxi. Input:
- xxxii. {"problem_id": "004", "initial_string": "buXfTof6OPg1umMKPqL#C4Dn", "transition s": [{"src": "ML#", "tgt": ""}, {"src": "C4Dn", "tgt": ""}, {"src": "buXOP", "tgt": ""}, {"src": "u m", "tgt": ""}, {"src": "fTof6", "tgt": ""}, {"src": "KPq", "tgt": ""}, {"src": "g1", "tgt": ""}, {"src": "wF7", "tgt": "I"}, {"src": "wF7", "tgt": ""}, {"src": "O1nn", "tgt": "xXK"}, {"src": "O1nn", "tgt": ""}, {"src": "O1nn", "tgt": "v5c"}, {"src": "u", "tgt": ""}, {"src": "u", "tgt": "4"}]}
- xxxiii.
- xxxiv. Reasoning:
- xxxv. - Current: buXfTof6OPg1umMKPqL#C4Dn
- xxxvi. - Apply transition 6 ("g1" → ""): buXfTof6OPumMKPqL#C4Dn
- xxxvii. - Apply transition 5 ("KPq" → ""): buXfTof6OPumML#C4Dn
- xxxviii. - Apply transition 4 ("fTof6" → ""): buXOPumML#C4Dn
- xxxix. - Apply transition 3 ("um" → ""): buXOPML#C4Dn
- xl. - Apply transition 2 ("buXOP" → ""): ML#C4Dn
- xli. - Apply transition 1 ("C4Dn" → ""): ML#
- xlii. - Apply transition 0 ("ML#" → ""): ""
- xliii. Final string is empty.
- xliv.
- xlv. Final Answer:
- xlvi. {"problem_id": "004", "solution": [6,5,4,3,2,1,0]}
- xlvi.
- xlvi. ---
- xlvi.
- l. ### Example 2
- li. Input:
- lii. {"problem_id": "016", "initial_string": "...MM..MM", "transitions": [{"src": ".", "tgt": ".."}, {"src": ".", "tgt": "."}, {"src": ".", "tgt": ""}, {"src": "M", "tgt": ".."}, {"src": "M", "tgt": "MM"}, {"src": "M", "tgt": ""}, {"src": "M", "tgt": "M"}]}
- liii.
- liv. Reasoning:
- lv. - Current: ...MM..MM
- lvi. - Apply 5 ("M" → ""): ..M..MM
- lvii. - Apply 2 ("." → ""): .M..MM
- lviii. - Apply 5 ("M" → ""): . ..MM

- lix. - Apply 2 (". "→""): ..MM
- lx. - Apply 5 ("M"→""): .M
- lxi. - Apply 2 (". "→""): M
- lxii. - Apply 2 (". "→""): (no effect, still M)
- lxiii. - Apply 2 (". "→""): (still M)
- lxiv. - Apply 5 ("M"→""): ""
- lxv. Final string is empty.
- lxvi.
- lxvii. Final Answer:
- lxviii. {"problem_id": "o16", "solution": [5, 2, 5, 2, 5, 2, 2, 5]}
- lxix.
- lxx. ---
- lxxi.
- lxxii. ### Example 3
- lxxiii. Input:
- lxxiv. {"problem_id": "o26", "initial_string": "2SLHhMKHhMo2SLKHhM", "transitions": [{"src": "HhM", "tgt": "2SL"}, {"src": "2SL", "tgt": "K"}, {"src": "K", "tgt": "oo"}, {"src": "o", "tgt": ""}]}
- lxxv.
- lxxvi. Reasoning:
- lxxvii. - Current: 2SLHhMKHhMo2SLKHhM
- lxxviii. - Apply 1 ("2SL"→"K"): KHhMKHhMo2SLKHhM
- lxxix. - Apply 2 ("K"→"oo"): ooHhMKHhMo2SLKHhM
- lxxx. - Apply 3 ("o"→""): oHhMKHhMo2SLKHhM
- lxxxi. - Apply 3 ("o"→""): HhMKHhMo2SLKHhM
- lxxxii. - Apply 0 ("HhM"→"2SL"): 2SLKHhMo2SLKHhM
- lxxxiii. - Apply 1 ("2SL"→"K"): KKHhMo2SLKHhM
- lxxxiv. - Apply 2 ("K"→"oo"): ooKHhMo2SLKHhM
- lxxxv. - Apply 3 ("o"→""): oKHhMo2SLKHhM
- lxxxvi. - Apply 3 ("o"→""): KHhMo2SLKHhM
- lxxxvii. - Apply 0 ("HhM"→"2SL"): K2SLo2SLKHhM
- lxxxviii. ... (continue steps until string is empty)
- lxxxix.
- xc. Final Answer:
- xc. {"problem_id": "o26", "solution": [1, 2, 3, 3, 0, 1, 2, 3, 3, 2, 3, 3, 0, 1, 2, 3, 3, 3, 1, 2, 3, 3, 2, 3, 3, 0, 1, 2, 3, 3]}

- Now model gave **correct answer solution**



- Some more questions tested from easy dataset: That was **correct** too



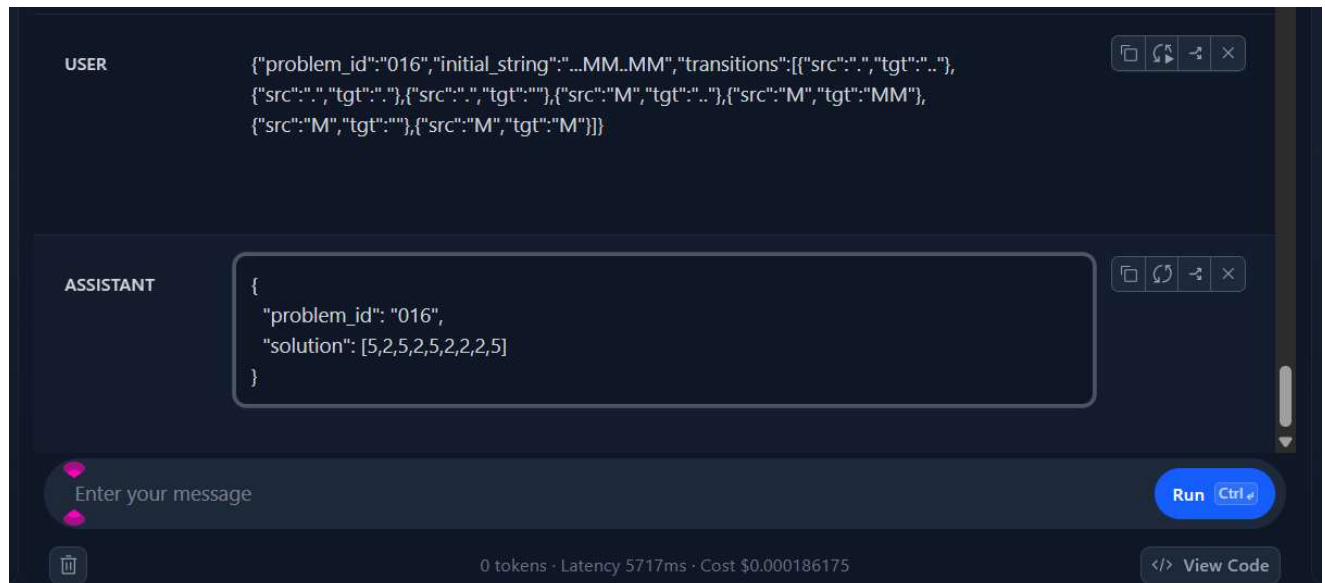
The screenshot shows a chat interface with a dark theme. The 'USER' section contains a JSON object with a problem_id of '048', an initial_string of '0e7b5oht', and a set of transitions. The 'ASSISTANT' section returns a JSON object with the same problem_id and a solution of [2, 0, 1]. At the bottom, there is a text input field labeled 'Enter your message' and a blue 'Run' button with a keyboard shortcut 'Ctrl ↵'.

```
USER {"problem_id":"048","initial_string":"0e7b5oht","transitions":[{"src":"0b","tgt":""}, {"src":"5oht","tgt":""}, {"src":"e7","tgt":""}]}
```

```
ASSISTANT {  "problem_id": "048",  "solution": [2, 0, 1]}
```

Enter your message Run Ctrl ↵

- Now let's test the medium dataset: Which was **accurately** solved as well
- Let's try harder dataset: Here it hallucinated however the answer string was too long as well.



The screenshot shows a chat interface similar to the first one. The 'USER' section contains a JSON object with a problem_id of '016', an initial_string of '...MM..MM', and a set of transitions. The 'ASSISTANT' section returns a JSON object with the same problem_id and a solution of [5, 2, 5, 2, 5, 2, 2, 5]. At the bottom, there is a text input field labeled 'Enter your message', a blue 'Run' button with a keyboard shortcut 'Ctrl ↵', and a footer showing '0 tokens · Latency 5717ms · Cost \$0.000186175' and a 'View Code' button.

```
USER {"problem_id":"016","initial_string":"...MM..MM","transitions":[{"src":"","tgt":".."}, {"src":"","tgt":"."}, {"src":"","tgt":""}, {"src":"M","tgt":".."}, {"src":"M","tgt":"MM"}, {"src":"M","tgt":""}, {"src":"M","tgt":"M"}]}
```

```
ASSISTANT {  "problem_id": "016",  "solution": [5,2,5,2,5,2,2,5]}
```

Enter your message Run Ctrl ↵

0 tokens · Latency 5717ms · Cost \$0.000186175 View Code


```

USER      {"problem_id": "019", "initial_string": "mKlmKleTixsfcq-ieeTixsfqcmKl", "transitions":
          [{"src": "eTi", "tgt": "-ie"}, {"src": "-ie", "tgt": "mKlmKl"}, {"src": "mKl", "tgt": "cq"},
          {"src": "cq", "tgt": "xsf"}, {"src": "xsf", "tgt": ""}]}

ASSISTANT {
  "problem_id": "019",
  "solution": [0, 1, 2, 3, 4]
}

```

0 tokens · Latency 2650ms · Cost \$0.000186175

- Tried one more harder question: Again it couldn't traverse the tokens and hallucinated. The

```

USER      {"problem_id": "048", "initial_string": "j6%mb%b%O_j6%b%mm", "transitions":
          [{"src": "m", "tgt": "j6%"}, {"src": "j6%", "tgt": "O_"}, {"src": "O_", "tgt": "b%b%"}, {"src": "b%", "tgt": ""}]}

ASSISTANT {
  "problem_id": "048",
  "solution": [1, 0, 2, 3]
}

```

0 tokens · Latency 2673ms · Cost \$0.000186175

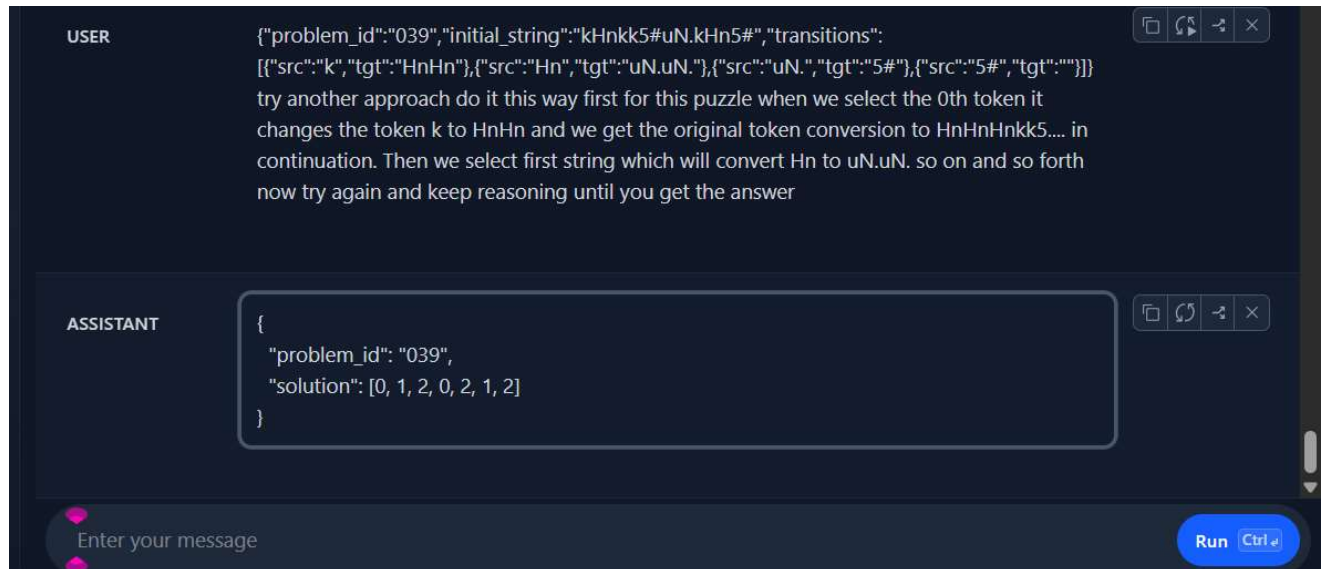
answer isn't correct for this hard problem as well.

- This proved that when easy and medium puzzles can be solved, but it hallucinate in traversing the tokens and cannot generate longer thinking and reasoning skills, which is much proven fact about smaller models.
8. Now let's use **Self-Consistency prompting** where we try to explicitly help the model to solve the harder traversing problems and help them understand how it is solved (I will only give harder problems in this model no trials for easier and medium level questions)
- The prompt which I use for self-Consistency is:
 - "SELF-CONSISTENCY PROMPT"*
 -
 - You will be given a string-rewrite puzzle in JSON. Each puzzle contains:*
 - "problem_id": string id*
 - "initial_string": starting string*
 - "transitions": ordered list of objects {"src": "...", "tgt": "..."}"*
 -
 - Goal:*

- ix. Reduce the "initial_string" to the empty string "" by applying transitions repeatedly.
- x.
- xi. Method (what you must follow):
- xii. 1. At each step, pick one transition index (0-based).
- xiii. 2. Locate the FIRST (leftmost) occurrence of the transition's "src" in the current string.
 - xiv. - If "src" is the empty string, treat it as matching at position 0.
- xv. 3. Replace that single occurrence with the transition's "tgt".
- xvi. 4. Repeat steps 1–3 until the string becomes "".
- xvii. 5. The solution is the ordered list of transition indices you applied.
- xviii.
- xix. Self-consistency rule:
- xx. - Internally try at least 3 different plausible sequences of transitions (different choices when multiple matches exist).
- xxi. - Select the first sequence that successfully reduces the string to "" and return that sequence.
- xxii. - If none of the internal attempts succeed, return an empty list: [].
- xxiii.
- xxiv. Strict output rules (READ CAREFULLY):
- xxv. - OUTPUT EXACTLY ONE JSON OBJECT and NOTHING ELSE (no explanation, no code, no extra text).
- xxvi. - JSON format must be exactly:
- xxvii.
- xxviii. {
 - xxix. "problem_id": "<same id as input>",
 - xxx. "solution": [list of 0-based indices]
 - xxxi. }
 - xxxii.
 - xxxiii. - If unsolvable, put "solution": [].
 - xxxiv.
 - xxxv. HARD EXAMPLES (Input -> Output pairs). These show expected final JSON only:
 - xxxvi.
 - xxxvii. Example A
 - xxxviii. Input:
 - xxxix. {"problem_id": "o48", "initial_string": "j6%mb%b%O_j6%b%mm", "transitions": [{"src": "m", "tgt": "j6%", {"src": "j6%", "tgt": "O_"}, {"src": "O_", "tgt": "b%b%"}, {"src": "b%", "tgt": ""}]}
 - xl. Output:
 - xli. {"problem_id": "o48", "solution": [1,2,3,3,0,1,2,3,3,3,2,3,3,1,2,3,3,0,1,2,3,3,0,1,2,3,3]}
 - xlii.
 - xliii. Example B
 - xliv. Input:
 - xlv. {"problem_id": "o24", "initial_string": "kn1sKesKe&G&G&Gn1sKe", "transitions": [{"src": "n1", "tgt": "s"}, {"src": "s", "tgt": "&G&G"}, {"src": "&G", "tgt": "sKe"}, {"src": "sKe", "tgt": "kk"}, {"src": "k", "tgt": ""}]}
 - xlvi. Output:
 - xlvii. {"problem_id": "o24", "solution": [4,0,1,2,3,4,4,2,3,4,4,3,4,4,3,4,4,2,3,4,4,2,3,4,4,0,1,2,3,4,4,2,3,4,4,3,4,4]}
 - xlviii.
 - xl ix. Example C
 - l. Input:
 - li. {"problem_id": "o26", "initial_string": "2SLHhMKHhMo2SLKHhM", "transitions": [{"src": "HhM", "tgt": "2SL"}, {"src": "2SL", "tgt": "K"}, {"src": "K", "tgt": "oo"}, {"src": "o", "tgt": ""}]}
 - lii. Output:

- liii. `{"problem_id": "026", "solution": [1, 2, 3, 3, 0, 1, 2, 3, 3, 2, 3, 3, 0, 1, 2, 3, 3, 1, 2, 3, 3, 2, 3, 3, 0, 1, 2, 3, 3]}`
- liv.
- lv. Now solve the following puzzle below. Remember:
- lvi. - Internally explore multiple paths (at least 3).
- lvii. - Pick the first path that yields an empty string.
- lviii. - Do not output internal reasoning—only the final JSON result as defined above.”
- I paired this with Temperature = 0.7 (higher temperature) and Top_P = 0.9 (for diverse sampling and more method variety).
- Then I ran the model with same question for 10 times and repeated this for 3 questions in total. (N sampling = 10 times for 3 questions)
- After few passes I again gave this prompt:
 - i. `V{"problem_id": "039", "initial_string": "kHnkk5#uN.kHn5#", "transitions": [{"src": "k", "tgt": "HnHn"}, {"src": "Hn", "tgt": "uN.uN."}, {"src": "uN.", "tgt": "5#"}, {"src": "5#", "tgt": ""}]}` try another approach do it this way first for this puzzle when we select the 0th token it changes the token k to HnHn and we get the original token conversion to HnHnHnkk5.... in continuation. Then we select first string which will convert Hn to uN.uN. so on and so forth now try again and keep reasoning until you get the answer”

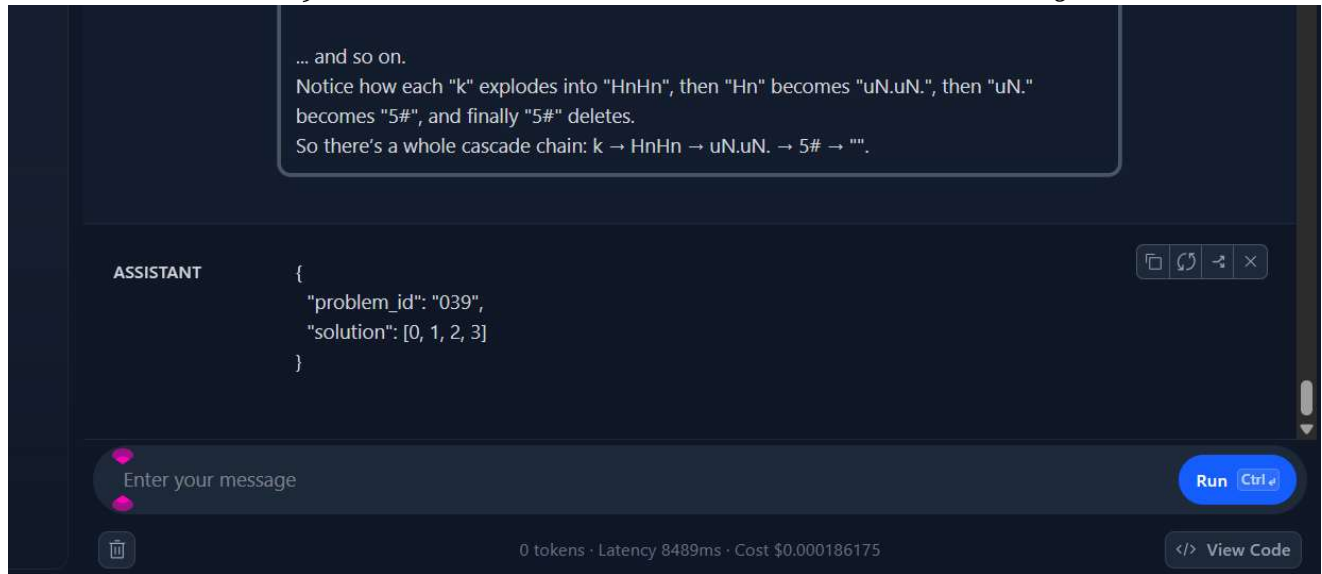
To which it again hallucinated and couldn't give correct answer



- ii. Then I tried prompting it like this: for an specific example and this is what I passed on:

1. "Puzzle:
2. `initial_string = "kHnkk5#uN.kHn5#"`
3. transitions:
4. 0: "k" → "HnHn"
5. 1: "Hn" → "uN.uN."
6. 2: "uN." → "5#"
7. 3: "5#" → ""
8. Let's begin:
9. `Start = kHnkk5#uN.kHn5#.`
10. `Leftmost k → apply rule 0 → "HnHnHnkk5#uN.kHn5#".`
11. `Now the leftmost piece is "Hn" → apply rule 1 →`
12. `"uN.uN.HnHnkk5#uN.kHn5#".`
13. `Again "uN." appears → apply rule 2 →`
14. `"5#uN.HnHnkk5#uN.kHn5#".`
15. `Now leftmost "5#" → apply rule 3 →`
16. `"uN.HnHnkk5#uN.kHn5#".`
17. ... and so on.

18. Notice how each "k" explodes into "HnHn", then "Hn" becomes "uN.uN.", then "uN." becomes "5#", and finally "5#" deletes.
19. So there's a whole cascade chain: $k \rightarrow HnHn \rightarrow uN.uN. \rightarrow 5\# \rightarrow ""$



- iii. To which it generated a short string which was correct till the point it generated but it was just string omission and not complete string traversing method.
- iv. Then I went on and EXPLICITLY explained how to solve the puzzle with this prompt:

1. **Puzzle (ID 039)**

2. **Initial string:**

"kHnkk5#uN.kHn5#"

3. **Transitions (index → rule):**

4. 0: "k" → "HnHn"

5. 1: "Hn" → "uN.uN."

6. 2: "uN." → "5#"

7. 3: "5#" → ""

8. (Recall: at each step we apply `current.replace(src, tgt, 1)` i.e. replace only the leftmost occurrence of src.)

9.

10. **Full step-by-step solution (Level 3)**

11. I list each step as:

12. Step N — apply rule <index> (<src> → <tgt>) : "before" → "after"

13. Step 1 — apply rule 0 (k → HnHn):

"kHnkk5#uN.kHn5#" → "HnHnHnkk5#uN.kHn5#"

14. Step 2 — apply rule 1 (Hn → uN.uN.):

"HnHnHnkk5#uN.kHn5#" → "uN.uN.HnHnkk5#uN.kHn5#"

15. Step 3 — apply rule 2 (uN. → 5#):

"uN.uN.HnHnkk5#uN.kHn5#" → "5#uN.HnHnkk5#uN.kHn5#"

16. Step 4 — apply rule 3 (5# → ""):

"5#uN.HnHnkk5#uN.kHn5#" → "uN.HnHnkk5#uN.kHn5#"

17. Step 5 — apply rule 2 (uN. → 5#):

"uN.HnHnkk5#uN.kHn5#" → "5#HnHnkk5#uN.kHn5#"

18. Step 6 — apply rule 3 (5# → ""):

"5#HnHnkk5#uN.kHn5#" → "HnHnkk5#uN.kHn5#"

19. Step 7 — apply rule 1 (Hn → uN.uN.):

"HnHnkk5#uN.kHn5#" → "uN.uN.Hnkk5#uN.kHn5#"

20. Step 8 — apply rule 2 (uN. → 5#):

"uN.uN.Hnkk5#uN.kHn5#" → "5#uN.Hnkk5#uN.kHn5#"

21. Step 9 — apply rule 3 (5# → ""):

"5#uN.Hnkk5#uN.kHn5#" → "uN.Hnkk5#uN.kHn5#"

22. Step 10 — apply rule 2 (uN. -> 5#):
"uN.Hnkk5#uN.kHn5#" → "5#Hnkk5#uN.kHn5#"
23. Step 11 — apply rule 3 (5# -> ""):
"5#Hnkk5#uN.kHn5#" → "Hnkk5#uN.kHn5#"
24. Step 12 — apply rule 1 (Hn -> uN.uN.):
"Hnkk5#uN.kHn5#" → "uN.uN.kk5#uN.kHn5#"
25. Step 13 — apply rule 2 (uN. -> 5#):
"uN.uN.kk5#uN.kHn5#" → "5#uN.kk5#uN.kHn5#"
26. Step 14 — apply rule 3 (5# -> ""):
"5#uN.kk5#uN.kHn5#" → "uN.kk5#uN.kHn5#"
27. Step 15 — apply rule 2 (uN. -> 5#):
"uN.kk5#uN.kHn5#" → "5#kk5#uN.kHn5#"
28. Step 16 — apply rule 3 (5# -> ""):
"5#kk5#uN.kHn5#" → "kk5#uN.kHn5#"
29. Step 17 — apply rule 0 (k -> HnHn):
"kk5#uN.kHn5#" → "HnHnk5#uN.kHn5#"
30. Step 18 — apply rule 1 (Hn -> uN.uN.):
"HnHnk5#uN.kHn5#" → "uN.uN.Hnk5#uN.kHn5#"
31. Step 19 — apply rule 2 (uN. -> 5#):
"uN.uN.Hnk5#uN.kHn5#" → "5#uN.Hnk5#uN.kHn5#"
32. Step 20 — apply rule 3 (5# -> ""):
"5#uN.Hnk5#uN.kHn5#" → "uN.Hnk5#uN.kHn5#"
33. Step 21 — apply rule 2 (uN. -> 5#):
"uN.Hnk5#uN.kHn5#" → "5#Hnk5#uN.kHn5#"
34. Step 22 — apply rule 3 (5# -> ""):
"5#Hnk5#uN.kHn5#" → "Hnk5#uN.kHn5#"
35. Step 23 — apply rule 1 (Hn -> uN.uN.):
"Hnk5#uN.kHn5#" → "uN.uN.k5#uN.kHn5#"
36. Step 24 — apply rule 2 (uN. -> 5#):
"uN.uN.k5#uN.kHn5#" → "5#uN.k5#uN.kHn5#"
37. Step 25 — apply rule 3 (5# -> ""):
"5#uN.k5#uN.kHn5#" → "uN.k5#uN.kHn5#"
38. Step 26 — apply rule 2 (uN. -> 5#):
"uN.k5#uN.kHn5#" → "5#k5#uN.kHn5#"
39. Step 27 — apply rule 3 (5# -> ""):
"5#k5#uN.kHn5#" → "k5#uN.kHn5#"
40. Step 28 — apply rule 0 (k -> HnHn):
"k5#uN.kHn5#" → "HnHn5#uN.kHn5#"
41. Step 29 — apply rule 1 (Hn -> uN.uN.):
"HnHn5#uN.kHn5#" → "uN.uN.Hn5#uN.kHn5#"
42. Step 30 — apply rule 2 (uN. -> 5#):
"uN.uN.Hn5#uN.kHn5#" → "5#uN.Hn5#uN.kHn5#"
43. Step 31 — apply rule 3 (5# -> ""):
"5#uN.Hn5#uN.kHn5#" → "uN.Hn5#uN.kHn5#"
44. Step 32 — apply rule 2 (uN. -> 5#):
"uN.Hn5#uN.kHn5#" → "5#Hn5#uN.kHn5#"
45. Step 33 — apply rule 3 (5# -> ""):
"5#Hn5#uN.kHn5#" → "Hn5#uN.kHn5#"
46. Step 34 — apply rule 1 (Hn -> uN.uN.):
"Hn5#uN.kHn5#" → "uN.uN.5#uN.kHn5#"
47. Step 35 — apply rule 2 (uN. -> 5#):
"uN.uN.5#uN.kHn5#" → "5#uN.5#uN.kHn5#"
48. Step 36 — apply rule 3 (5# -> ""):
"5#uN.5#uN.kHn5#" → "uN.5#uN.kHn5#"
49. Step 37 — apply rule 2 (uN. -> 5#):
"uN.5#uN.kHn5#" → "5#5#uN.kHn5#"

50. Step 38 — apply rule 3 (5# -> ""):
"5#5#uN.kHn5#" → "5#uN.kHn5#"
51. Step 39 — apply rule 3 (5# -> "") again (leftmost 5#):
"5#uN.kHn5#" → "uN.kHn5#"
52. Step 40 — apply rule 2 (uN. -> 5#):
"uN.kHn5#" → "5#kHn5#"
53. Step 41 — apply rule 3 (5# -> ""):
"5#kHn5#" → "kHn5#"
54. Step 42 — apply rule 0 (k -> HnHn):
"kHn5#" → "HnHnHn5#"
55. Step 43 — apply rule 1 (Hn -> uN.uN.):
"HnHnHn5#" → "uN.uN.HnHn5#"
56. Step 44 — apply rule 2 (uN. -> 5#):
"uN.uN.HnHn5#" → "5#uN.HnHn5#"
57. Step 45 — apply rule 3 (5# -> ""):
"5#uN.HnHn5#" → "uN.HnHn5#"
58. Step 46 — apply rule 2 (uN. -> 5#):
"uN.HnHn5#" → "5#HnHn5#"
59. Step 47 — apply rule 3 (5# -> ""):
"5#HnHn5#" → "HnHn5#"
60. Step 48 — apply rule 1 (Hn -> uN.uN.):
"HnHn5#" → "uN.uN.Hn5#"
61. Step 49 — apply rule 2 (uN. -> 5#):
"uN.uN.Hn5#" → "5#uN.Hn5#"
62. Step 50 — apply rule 3 (5# -> ""):
"5#uN.Hn5#" → "uN.Hn5#"
63. Step 51 — apply rule 2 (uN. -> 5#):
"uN.Hn5#" → "5#Hn5#"
64. Step 52 — apply rule 3 (5# -> ""):
"5#Hn5#" → "Hn5#"
65. Step 53 — apply rule 1 (Hn -> uN.uN.):
"Hn5#" → "uN.uN.5#"
66. Step 54 — apply rule 2 (uN. -> 5#):
"uN.uN.5#" → "5#uN.5#"
67. Step 55 — apply rule 3 (5# -> ""):
"5#uN.5#" → "uN.5#"
68. Step 56 — apply rule 2 (uN. -> 5#):
"uN.5#" → "5#5#"
69. Step 57 — apply rule 3 (5# -> ""):
"5#5#" → "5#"
70. Step 58 — apply rule 3 (5# -> "") again (last 5#):
"5#" → "" (EMPTY — puzzle solved)

71.

72. Formal explanation / pattern summary

73. Each *k* triggers a whole expansion cascade:

74. $k \rightarrow HnHn$

75. $Hn \rightarrow uN.uN.$

76. $uN. \rightarrow 5\#$

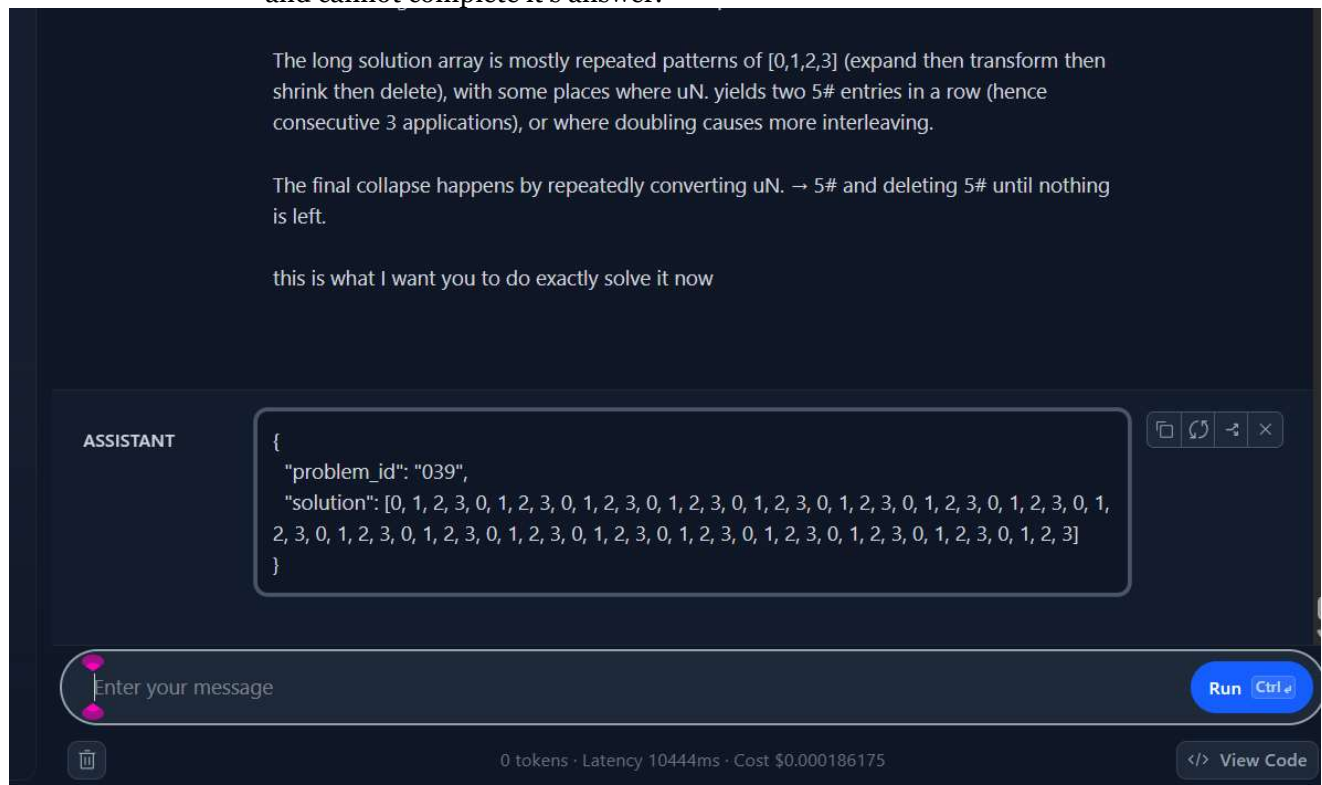
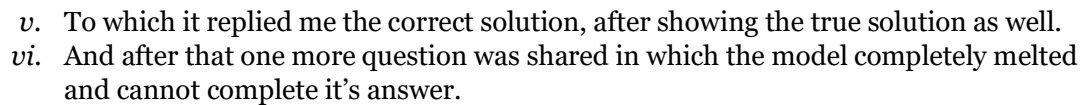
77. $5\# \rightarrow ""$ (deletes)

78. So a single *k* leads to a chain of **four** transformation steps before its contributions fully disappear — but because tokens can expand to multiple tokens, those steps interleave across the string.

79. The solver always targets the **leftmost match** at every step. That makes the process deterministic given the same transition-index sequence.

80. The long solution array is mostly repeated patterns of [0,1,2,3] (expand then transform then shrink then delete), with some places where *uN.* yields two

81. The final collapse happens by repeatedly converting $uN. \rightarrow 5\#$ and deleting $5\#$ until nothing is left.”



9. This Brings us to the conclusion that I was not able to solve harder puzzles by even Self-Consistency prompting style.

10. Summary:

- Zero-Shot prompting: Couldn't even solve simpler problems from my dataset
- Few-Shot prompting: Too couldn't even solve simpler problems from my dataset
- Chain-of-Thoughts prompting: was able to solve easy and medium level of puzzles very accurately but not the hard puzzles
- Self-Consistency Prompting too was not able to solve the harder puzzles from my dataset at all.

Task 3 Evaluating Responses

1. Validity Score (V):

- If the generated response solves the puzzle or not:
 - $V = 0$; if not solved
 - $V = 1$; if solved correctly

2. Efficiency Score (E):

- L_{llm} = length of solution produced by LLM
- L_{ts} = length of true solution

$$i. E = \frac{L_{ts}}{L_{llm}}$$

- If $E = 1$; Model is accurate
- If $0 < E < 1$ || $E > 1$; LLM couldn't reason it complete solution
- If $E = 0$; LLM wasn't able to solve at all

3. Consistency Score (C):

$$a. C = \frac{\text{Number of valid runs}}{\text{\# of total runs}}$$

- If $C = 1$; The prompt was perfect and LLM works perfectly
- If $C < 1$; LLM works less accurately in forward passes than total number of forward passes.

4. Energy Level with difficulty of questions (s_d)

$$a. \bar{S}_D = \frac{D_i \cdot S_{\text{puzzle},i}}{\sum_i D_i}$$

- $S_{\text{puzzle},i}$ = Average of the puzzle solved by the model
 - D_i = Difficulty level for distinct question
 - $\sum_i D_i$ = Difficulty level summation
- This will help us calculating the model effectiveness by varying difficulty of the puzzle, which intuitively means we are taking varying difficulty (D_i) into account before penalizing the 8b parameters model.

Task 4 Human vs Machine

Tasks which stand easy for LLMs, but tough for Humans:

- Memorisation task, referring copious amount of text and drawing conclusion (to some extent) are easier for LLMs but tough for humans, because of the limited amount of the storage and compute power of human brains. This limitation does not hold true for LLMs and Machines.
- Inherent reasoning (like our puzzles solution for example) and everyday commonsense (like to use a crinkle knife for salad when serving to impress, which cannot be detected by llms unless stated explicitly or implicitly from learnt pattern) are some tasks which can be done way better by humans, but LLMs lack to catch upto the human capability levels.